Duplicate Finder

Code guide — how it works inside

A tour of the code for anyone who wants to understand it or learn from it: the layered architecture with ports and adapters, the algorithms (the 3 passes of the scan, dHash, BK-tree, Union-Find), the interface's threading pattern, the languages, the packaging and how everything is tested.

Desktop application for Windows · Python + CustomTkinter
Document generated on 2026-10-24

Contents

1. How to read this project 3

1.1. Module map 3

2. The logic, explained 5

2.1. Detecting exact duplicates: three passes 5

2.2. The hash: BLAKE2b and block reading 5

2.3. The filter: whitelist and blacklist 6

2.4. The decision: which copy is kept 7

2.5. Similar photos: dHash + BK-tree + Union-Find 7

2.6. Moving files safely 8

2.7. History and undo 8

3. The architecture: ports, adapters and composition root 9

4. The interface, explained 10

4.1. CustomTkinter and startup 10

4.2. The thread + queue + after() pattern (the most important one) 10

4.3. The three tabs 10

4.4. Texts and languages 10

4.5. Light / dark theme 11

4.6. Simple / Advanced view 11

4.7. The table (ttk.Treeview) 11

4.8. Thumbnails and images in Tkinter 11

5. Packaging 12

6. How it is tested 12

7. Adding a new feature: a recipe 12

1. How to read this project

This guide explains how the code is built, so that you can understand it and learn from it, just as I have been learning. It lives in the `src/duplicate_finder/` package, split into layers, with a test folder next to it:

Folder	What it contains	Rule
domain/	Rules and data: filters, duplicate groups, selection, similarity.	Imports nothing from the rest. No input or output (no disk, no window).
application/	The use cases (scan, move, gather, history...) and the ports they need.	Imports only domain/ and i18n/. The disk only through ports.
infrastructure/	The adapters: disk, hashes, Pillow, JSON, logging.	Implements the ports. Never imports the interface.
presentation/	The window and all its widgets (CustomTkinter).	Only draws and reacts to clicks; delegates to the use cases.
i18n/	The English and Spanish texts.	Imports nothing.
tests/	pytest tests, at several levels.	They create real files or use in-memory fakes of the ports.

Where I recommend you start reading: `application/scan_duplicates.py` (the central algorithm), then `bootstrap.py` (how everything is wired) and `presentation/main_window.py` (the window shell; each tab is a component in its own `*_tab.py`). The rest are pieces that explain themselves with their docstring.

1.1. Module map

domain/	
duplicates.py	DuplicateFile, DuplicateGroup, ScanResult: the result of a scan.
filters.py	ScanFilter: what to include/exclude. CATEGORY_EXTENSIONS.
selection.py	DuplicateSelection: keep/move state per group.
similarity.py	similar photos: Hamming distance, BK-tree, Union-Find.
transfers.py	move/copy rules and what to do with the non-image files.
formatting.py, operations.py	readable sizes (KB/MB) and operation kinds.
application/	
ports.py	the interfaces (Protocol): FileSystem, FileHasher, FingerprintReader, HistoryStore, SettingsStore.
scan_duplicates.py	ScanDuplicates: the 3 passes. Returns a ScanResult.
find_similar_photos.py	FindSimilarPhotos: fingerprints + grouping.
quarantine.py	MoveToQuarantine: validate, move preserving structure, CSV, history.
gather.py, file_transfers.py	GatherImages and the batch move/copy primitives.
history.py	OperationHistory: log of operations + undo.
report.py, settings.py, start_selection.py	CSV export, settings, preselection.

application/	
services.py	Services: all the use cases together, for the interface.
infrastructure/	
local_file_system.py	LocalFileSystem: os and shutil (with retries on delete).
hashing.py	Blake2Hasher: partial and full BLAKE2b hash.
image_fingerprints.py	PillowFingerprintReader: dHash and colour signature.
history_store.py, settings.py	JsonHistoryStore and JsonSettingsStore (and the migration of the 1.0 data).
app_logging.py, data_folder.py	error log and data folder.
presentation/	
main_window.py	MainWindow: header, theme, language and view. Creates the 3 tabs.
exact_duplicates_tab.py	ExactDuplicatesTab: the Exact duplicates tab.
similar_photos_tab.py	SimilarPhotosTab: the Similar photos tab.
gather_images_tab.py	GatherImagesTab: the Gather images tab.
scan_options_panel.py, preview_panel.py	the two panels of the first tab.
background_job.py	BackgroundJob: thread + queue + polling, for every long task.
results_table.py	ResultsTable: the table of exact duplicates.
dialogs.py	the questions the three tabs share when moving.
image_viewer.py, history_window.py, ...	viewer with zoom, history + Undo, dialogs.
thumbnails.py, widgets.py, styles.py, icons.py	thumbnails, visual pieces, light/dark palette and glyphs.

2. The logic, explained

2.1. Detecting exact duplicates: three passes

Comparing the content of every file with that of every other would be painfully slow. Instead I filter from cheap to expensive (`application/scan_duplicates.py`). Each pass is a method (`_size_pass`, `_partial_hash_pass` and `_full_hash_pass`) and `ScanDuplicates.__call__()` only chains them:

- **Pass 1 — sizes.** I walk the folder with the `FileSystem.walk` port (underneath, `os.walk`) and group the files by size in bytes. A unique size cannot have duplicates, so I discard it without opening the file.
- **Pass 2 — partial hash.** For those that share a size, I compute the BLAKE2b hash of only the first 64 KB. Fast, and it discards almost all of those that are not equal.
- **Pass 3 — full hash.** Only for those that also share a partial hash, I hash the whole file. Those that match here are identical byte for byte.

```
# key = (size, partial_hash): two files with the same partial hash
# but a different size are never the same file.
by_partial[(size, hp)].append(path)
...
finalists = {key: paths for key, paths in by_partial.items() if len(paths) > 1}
```

Note. A subtle optimisation: if a file weighs less than 64 KB, its partial hash already covers the whole content, so I skip Pass 3 for it.

Pruning folders while walking

The walk hands you three things per folder: (path, subfolders, files). If you modify the list of subfolders *in place*, it does not enter the ones you remove — that is how `node_modules`, `$RECYCLE.BIN`, etc. are excluded without walking them:

```
subdirs[:] = [d for d in subdirs if not filter.excludes_folder(d)]
```

2.2. The hash: BLAKE2b and block reading

I use BLAKE2b (it ships with Python, faster than SHA-256). It lives in `infrastructure/hashing.py`, and the use case receives it as a port (`FileHasher`). I read the whole file 64 KB at a time so it is never loaded into memory in full:

```
h = hashlib.blake2b()
with open(file_path, 'rb') as f:
    while True:
        block = f.read(65536)      # 64 KB
        if not block:              # b'' = end of file
            break
        h.update(block)            # accumulates
return h.hexdigest()
```

2.3. The filter: whitelist and blacklist

`ScanFilter` (in `domain/filters.py`) has two mechanisms. A **whitelist** (`included_extensions`): if it is not empty, only those extensions pass ('search only images'). And several **blacklists** (extensions, folders and names to ignore, plus a minimum size). The scan calls `filter.excludes_name(...)` and `filter.excludes_folder(...)` and neither counts nor reads what is excluded.

2.4. The decision: which copy is kept

`DuplicateSelection` (in `domain/selection.py`) is a dictionary `(group_index, path) -> 'keep' | 'move'`. Everything starts as 'keep' (nothing would be moved). The rule the app enforces: to be able to move, every group must have **exactly one** copy set to 'keep'.

In Simple view, `preselect_primary()` picks the main copy of each group by itself with this score (the smallest tuple wins):

```
(looks_like_copy,  # 0 if the name has no 'copy', '(1)'...  1 if it does
 depth,            # number of folders: the less nested, the better
 mtime,            # date: the oldest is usually the original
 len(path))        # if all else is equal, the shortest path
```

2.5. Similar photos: dHash + BK-tree + Union-Find

Here I cannot compare content (two similar photos have different bytes). For each image I compute a **fingerprint** (`infrastructure/image_fingerprints.py`, with Pillow); comparing fingerprints is pure logic (`domain/similarity.py`):

64-bit dHash

I shrink the image to a 9×8 grayscale thumbnail. For each row, I compare each pixel with the one to its right: bit 1 if the brightness goes up, 0 if it goes down. 8×8 = 64 bits. It measures the *shape*, not the colour, which is why it withstands recompression and filters:

```
bits = 0
for row in range(8):
    for col in range(8):
        goes_up = gray[base+col] < gray[base+col+1]
        bits = (bits << 1) | (1 if goes_up else 0)
```

Hamming distance

How different two fingerprints are = how many bits differ. `bin(a ^ b).count('1')`. 0 = identical, 64 = opposite. The sensitivity slider sets the threshold (~10).

BK-tree

To avoid comparing every photo with every other, I put the dHashes in a BK-tree: a tree where each child is labelled with its exact distance to its parent. When searching for 'those within ≤ N', the triangle inequality lets me **prune** whole branches: I only have to look at the children whose label is in `[d-N, d+N]`.

Union-Find

If A looks like B and B like C, they must end up in the same group even if A and C do not look alike directly. Union-Find (disjoint sets) joins those chains: each element points to 'another one of its group' and following the chain leads to the representative. `groups()` groups by representative.

2.6. Moving files safely

The use cases that move or copy (`quarantine.py`, `file_transfers.py`, `gather.py`) share the same rules:

- **Never overwrite.** Before moving, if the destination already exists I try `name (1).ext`, `name (2).ext`... (`collision_free_path`).
- **Preserve structure.** The destination of each file is `destination_folder / relpath(source, root)`, so the subfolder hierarchy is recreated.
- **Move, not copy-and-delete.** `shutil.move` (inside the `LocalFileSystem` adapter) does an atomic `rename` on the same drive; it only copies+deletes across drives (and there the original disappears only once the copy is safe).
- **Destination outside the source.** `validate_destination` and `validate_quarantine_folder` check it (otherwise the next scan would find again what was moved).
- **Everything is recorded.** The use case itself writes the CSV log and the history entry; the interface does not have to remember to do it.

2.7. History and undo

Every move/copy operation is recorded (through the `HistoryStore` port, in `%APPDATA%\BuscadorDuplicados\history.json`) with the list of (source, destination) pairs. `OperationHistory.undo(id)`:

- if it was a **move**: it returns each file to its source — but if something already exists at the source, it warns and overwrites nothing.
- if it was a **copy**: it deletes the copies in the destination. It is the only point in the whole code where anything is deleted, and only files the app created. The deletion retries for a moment if Windows still has the file in use.

3. The architecture: ports, adapters and composition root

The trick that keeps all of the above tidy is that the use cases do not know *how* the disk is read, a hash is computed or a JSON is saved: they ask for those things through **ports**, which are interfaces (`typing.Protocol`) in `application/ports.py`:

```
class FileHasher(Protocol):
    partial_size: int
    def partial_hash(self, path: str) -> str: ...
    def full_hash(self, path: str) -> str: ...
```

A use case receives its ports in the constructor and never creates anything on its own:

```
class ScanDuplicates:
    def __init__(self, fs: FileSystem, hasher: FileHasher) -> None:
        self._fs = fs
        self._hasher = hasher
```

The **adapters** in `infrastructure/` implement those ports with the real world (`Blake2Hasher`, `LocalFileSystem`...). And `bootstrap.py` is the **composition root**: the only module that knows every layer. It builds the adapters, injects them and gives the window a `Services` object with all the use cases:

```
def build_services() -> Services:
    fs = LocalFileSystem()
    history = OperationHistory(JsonHistoryStore(), fs)
    return Services(
        scan_duplicates=ScanDuplicates(fs, Blake2Hasher()),
        find_similar_photos=FindSimilarPhotos(fs, PillowFingerprintReader()),
        move_to_quarantine=MoveToQuarantine(fs, history),
        ...)
```

What it buys

- **It is tested without a disk.** The use-case tests pass them in-memory fakes (`tests/fakes.py`): a file system that is a dictionary, a hasher that uses the content. They are fast and deterministic.
- **The interface could be replaced** without touching the logic, because it only sees `Services`.
- **Everything is in its place.** If you need to know how a file is moved, you look at an adapter; if you need to know when, you look at a use case.

Note. Not everything has a port: only what touches the outside world (disk, hashes, images, history, settings). Pure computations (filters, similarity, selection) are plain functions in `domain/`; an interface there would add nothing.

The rules, checked by a test

`tests/test_architecture.py` reads every module with `ast` and fails if a layer imports something it should not (for example `domain` importing `tkinter`, or `presentation` importing `infrastructure`), if the domain or the use cases call functions that touch the disk, or if anything outside `bootstrap.py` builds an adapter. The test itself is checked with code snippets that break each rule.

4. The interface, explained

4.1. CustomTkinter and startup

CustomTkinter is Tkinter with repainted widgets. `MainWindow` inherits from `ctk.CTk` and receives the `Services` object. `app.py` (or `python -m duplicate_finder`) calls `bootstrap.main()`: it starts the error log, builds the services and opens the window.

4.2. The thread + queue + after() pattern (the most important one)

Tkinter is not 'thread-safe': only the main thread can touch widgets. But scanning takes time. The solution lives in a single class, `presentation/background_job.py`, which every long task uses (scan, analyze, move, gather):

```
# main thread: launch. BackgroundJob schedules the polling with after().
self._scan_job.start(
    lambda report: self._services.scan_duplicates(folder, progress=report, ...),
    what=f'Scan of {folder}')

# inside BackgroundJob, worker thread: does NOT touch widgets, only the queue
result = work(lambda value: messages.put(('progress', value)))
messages.put(('done', result))

# main thread, every 100 ms: empty the queue and notify the tab
kind, datum = self._messages.get_nowait()
if kind == 'progress': self._on_progress(datum)
elif kind == 'done':   self._on_done(datum)
```

That way the window does not freeze, progress is visible and the task can be cancelled (the thread checks a `threading.Event` between files). If the task fails, `BackgroundJob` logs the error with its traceback and notifies the tab. It is tested without a window (`tests/test_background_job.py`) by replacing `after` with a scheduler the test drives.

4.3. The three tabs

A `ctk.CTkTabview` with three tabs. Each one is a **component**: a class that inherits from `ctk.CTkFrame` and keeps its OWN state (the result, the background tasks...) instead of sharing it with the window. The window only creates them (`self.exact`, `self.similar` and `self.gather`) and notifies them with `apply_mode(advanced)` and `cancel_jobs()`. Every tab repeats the pattern: a top card with the folder, a progress area, a statistics panel, a gallery/table and a bottom bar with the action button. The big parts are separate components (`ScanOptionsPanel`, `PreviewPanel`), and the move questions (choosing the quarantine, warnings) are in `dialogs.py`.

4.4. Texts and languages

No interface text is written in the code: each one is requested with `tr('key', ...)`, and the texts live in one catalog per language (`i18n/locales/en.py` and `es.py`), a dictionary from keys to sentences with placeholders:

```
"exact.moved_summary": "{moved} file(s) moved ({size})."           # en.py
"exact.moved_summary": "Se han movido {moved} archivo(s) ({size})."  # es.py

label.configure(text=tr('exact.moved_summary', moved=3, size='1.2 MB'))
```

The language is chosen at startup (the saved one, or Windows', or English), before any widget is created; that is why the header selector applies the next time. `tests/test_i18n.py` checks that both catalogs have the same keys and placeholders, and that every text the code asks for exists. The selectors (view, theme, strategy...) work with stable keys rather than the visible text, so changing language breaks nothing.

4.5. Light / dark theme

In `presentation/styles.py` each colour is a `(light, dark)` tuple. CustomTkinter widgets accept that tuple and change by themselves. For what is NOT CustomTkinter (the `ttk.Treeview` table and the viewer's `tk.Canvas`) I use `styles.col(tuple)`, which returns the colour of the active theme, and I repaint by hand by overriding `_set_appearance_mode`:

```
def _set_appearance_mode(self, mode_string):
    super()._set_appearance_mode(mode_string)
    self._configure_ttk_style()    # ttk does not change by itself
    self._configure_tags()
```

Disabled buttons turn grey (fill and text) with `styles.activate_button(btn, active)`, so that 'disabled' is not confused with an action or error colour. The chosen theme is saved with the rest of the settings (`AppSettings`).

4.6. Simple / Advanced view

`_apply_mode()` neither creates nor destroys widgets: it removes them from and puts them back in the grid with `grid_remove()` / `grid()`. That way the state (whatever the user had typed) is kept when toggling. The choice is saved in the settings.

4.7. The table (ttk.Treeview)

For exact duplicates a `ttk.Treeview` is used (it copes with thousands of rows). On a click you have to ask Tk what is under the mouse:

```
self.tree.identify_row(event.y)      # row id
self.tree.identify_column(event.x)   # '#1' = the 'Decision' column
self.tree.identify_region(x, y)      # 'heading', 'cell'...
```

With that: a normal click = select (and show the preview); a click on 'Decision' = toggle keep/move; a double-click = 'keep only this one'.

4.8. Thumbnails and images in Tkinter

- Decoding the image (Pillow) is done in a thread; it produces a PIL object.
- Converting to `ImageTk.PhotoImage` and putting it on the row is done in the main thread.
- **You must keep a reference** to each PhotoImage (`self._img_refs[iid] = photo`). Otherwise Python collects it and the image vanishes from the screen.

5. Packaging

`build.spec` configures PyInstaller: a single file, no console, with the CustomTkinter data included and `icon.ico` as the icon (the code lives in `src/`, which is added to the search path). `build_exe.py` runs it and checks the result.

The icon is a magnifying glass **drawn by code** (`presentation/icons.py:window_icon`) on a `tk.PhotoImage`, and `gen_icon.py` assembles the multi-size `.ico` by hand with `struct`. No external images.

6. How it is tested

There are several levels. The behaviour tests on files use no *mocks*: each one creates real files in a temporary folder (`tempfile.TemporaryDirectory`), runs the function for real and checks the state on disk. The use-case tests (`tests/test_use_cases.py`) use the in-memory fakes of `tests/fakes.py`. The interface tests build the whole window, simulate the dialogs (`filedialog` and `messagebox` are replaced by functions that return a fixed value) and press the buttons' methods. There are also tests for the architecture, for the language catalogs and for the migration of the data of version 1.0.

```
pytest                    # everything
pytest -m "not gui"       # logic only, no windows
pytest --cov              # with coverage (CI requires 90 % logic, 70 % interface)
python tests/check_perceptual.py "D:\Photos" 10   # with real data
```

7. Adding a new feature: a recipe

- **1.** Write the pure rule in `domain/` and the use case in `application/` (one class, with the ports it needs in the constructor). If it needs something from the outside world that does not exist yet, add a port and its adapter in `infrastructure/`.
- **2.** Wire it in `bootstrap.py` and add it to `Services`.
- **3.** Add tests: of the use case with in-memory fakes and, if it touches real files, with temporary folders.
- **4.** In `presentation/`, add the widgets and a handler that calls the use case. If the task is long, launch it with `BackgroundJob`. Every new text goes into both catalogs with `tr()`.
- **5.** If it moves or copies files, make it go through `FileTransfers` and record it in the history so it can be undone.
- **6.** Run `pytest`, `ruff check .` and `python -m mypy`. All green before considering anything done: including the architecture test, which warns you if you put something in the wrong layer.